

Feuille de TP n° 2

Bases de la programmation – Les fonctions

Rappels Python

Définir et appeler une fonction. Une fonction se définit avec **def**, ses paramètres entre parenthèses et un bloc indenté ; **return** renvoie une valeur à l'appelant et termine immédiatement la fonction :

```
def aire_rectangle(largeur, hauteur):    # définition
    aire = largeur * hauteur
    return aire                          # valeur renvoyée

a = aire_rectangle(3, 4)                 # appel : a vaut 12
print(aire_rectangle(2, 5))             # affiche 10
```

Renvoyer n'est pas afficher. **print** affiche à l'écran mais ne renvoie rien ; une fonction qui doit « donner » un résultat utilise **return**. Une fonction peut renvoyer un booléen ou plusieurs valeurs à la fois :

```
def est_pair(n):
    return n % 2 == 0                    # renvoie True ou False

def division(a, b):
    return a // b, a % b                 # renvoie deux valeurs

q, r = division(17, 5)                  # q vaut 3, r vaut 2
```

Exercice 1 Équation du second degré

On souhaite résoudre l'équation $ax^2 + bx + c = 0$ pour tous réels **a**, **b** et **c**, y compris lorsque **a** ou **b** sont nuls. La méthode est la suivante (*S* désigne l'ensemble des solutions) :

Entrée : trois réels *a*, *b* et *c*

Sortie : l'ensemble *S* des solutions de $ax^2 + bx + c = 0$

si *a* = 0 **alors**

si *b* = 0 **alors**

si *c* = 0 **alors** $S \leftarrow \mathbb{R}$ (tous les réels sont solutions)

sinon $S \leftarrow \emptyset$ (pas de solution)

sinon

$S \leftarrow \{-c/b\}$

sinon

$\Delta \leftarrow b^2 - 4ac$

si $\Delta = 0$ **alors** $S \leftarrow \{-b/(2a)\}$

sinon si $\Delta > 0$ **alors** $S \leftarrow \{\dots, \dots\}$ (à compléter)

sinon $S \leftarrow \emptyset$ (pas de solution réelle)

A. Écrire la fonction `discriminant(a, b, c)` qui renvoie le discriminant $\Delta = b^2 - 4ac$.

B. Écrire la fonction `resoudre(a, b, c)` qui résout l'équation. **Attention :** cette fonction ne doit

faire aucun affichage. Elle renvoie trois valeurs `nb`, `x1`, `x2` : `nb` est le nombre de solutions (-1 si tous les réels sont solutions, sinon 0 , 1 ou 2) et `x1`, `x2` sont les solutions éventuelles ($x1 < x2$ s'il y en a deux ; une valeur inutilisée vaut `None`).

- C. Écrire une fonction d'affichage, sans paramètre, qui demande `a`, `b` et `c` à l'utilisateur, appelle `resoudre` puis affiche le résultat de la résolution. Exemple d'exécution :

```
Entrez a : 0
Entrez b : 2
Entrez c : 1
Votre équation admet une seule solution : -0.5
```

Exercice 2 Décomposition en facteurs premiers

Voici la méthode de décomposition d'un entier en facteurs premiers.

Entrée : un entier naturel $n > 1$
Sortie : une chaîne de caractères qui représente la décomposition de n en facteurs premiers

```
resultat ← chaîne vide
tant que  $n > 1$  faire
     $d \leftarrow$  le plus petit diviseur premier de  $n$ 
    si resultat est vide alors
        | mettre  $d$  dans resultat
    sinon
        | ajouter " x " puis  $d$  à la fin de resultat
     $n \leftarrow n / d$  (division entière)
retourner resultat
```

- A. Écrire une fonction `est_premier(n)` qui renvoie `True` si n est un nombre premier, `False` sinon.
- B. Écrire une fonction `plus_petit_diviseur(n)` qui utilise `est_premier` et renvoie le plus petit diviseur premier de n (strictement supérieur à 1).
- C. Écrire une fonction `decomposition_facteurs(n)` qui produit la décomposition en facteurs premiers de n en appelant plusieurs fois la fonction de la question B. Par exemple, pour $n = 12$, la fonction renvoie la chaîne `"2 x 2 x 3"`.

Indication : ici, exceptionnellement, on utilise une variable de type `str` (chaîne de caractères). Au début, on écrit `resultat = ""`. Si `resultat` vaut à un moment `"2 x 2"` et que l'on veut ajouter l'entier d comme facteur, on écrit simplement `resultat = resultat + " x " + str(d)`.

Exercice 3 Le nombre mystère

On programme un jeu dans lequel l'utilisateur doit deviner, en un nombre limité de tentatives, un nombre entier tiré au hasard par l'ordinateur.

- A. Écrire une fonction `generer_nombre(minimum, maximum)` qui renvoie un entier aléatoire compris entre `minimum` et `maximum` (inclus). On utilisera `random.randint(minimum, maximum)`.
- B. Écrire une fonction `verifier_nombre(secret, proposition)` qui renvoie `1` si la proposition est plus petite que le nombre secret, `2` si elle est plus grande et `3` si elle est identique.
- C. Écrire une fonction `jouer()` qui demande à l'utilisateur combien de tentatives il souhaite, puis appelle `generer_nombre` pour créer le nombre mystère (entre 1 et 100). Ensuite, elle utilise une

boucle pour que l'utilisateur propose un nombre à chaque tentative ; elle utilise `verifier_nombre` pour lui donner un indice à chaque essai (« Le nombre mystère est plus grand » ou « Le nombre mystère est plus petit »). Le jeu se termine si l'utilisateur trouve le nombre ou si les tentatives sont épuisées ; la fonction affiche alors un message final avec le résultat (gagné, en indiquant le nombre de tentatives utilisées, ou perdu, en révélant le nombre mystère).

- D.** Écrire la fonction `main()` qui lance une partie puis demande, après chaque partie, à l'utilisateur s'il veut continuer à jouer. La fonction `main` sera appelée dans le programme comme ceci :

```
# Lancer le programme
if __name__ == "__main__":
    main()
```

Indication : la condition `__name__ == "__main__"` est vraie lorsque le fichier est exécuté directement (python3 `exo3.py`). Elle permet de placer l'appel initial à la fin du fichier, après toutes les définitions de fonctions.

Exercice 4 Factorielle, coefficient binomial et triangle de Pascal

- A.** Écrire une fonction `factorielle` qui prend en argument un entier naturel n et qui renvoie $n!$. Rappel : $n! = 1 \times 2 \times \dots \times n$; on ajoute $0! = 1$.
- B.** Écrire une fonction `coeff_binomial` qui prend en argument deux entiers naturels n et k (avec $k \leq n$) et qui renvoie le coefficient binomial correspondant, « k parmi n », donné par la formule

$$\binom{n}{k} = \frac{n!}{(n-k)!k!}.$$

Le résultat renvoyé doit être un entier.

- C.** Écrire une fonction `triangle_pascal` qui prend en argument un entier `nb_lignes` et qui affiche le triangle de Pascal avec `nb_lignes` lignes, les nombres d'une même ligne étant séparés par un espace. Voici un exemple avec 6 lignes :

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

En numérotant les lignes et les colonnes à partir de 0, le nombre situé sur la ligne numéro n et la colonne numéro k est le coefficient binomial $\binom{n}{k}$.

Une des applications du triangle de Pascal : les coefficients de la ligne numéro 4, soit 1 4 6 4 1, sont aussi les coefficients du développement

$$(a + b)^4 = a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4.$$